Task 1: Optimizer Performance on Non-Convex Functions

1. Objectives

- Implement the Rosenbrock and simplified 1D Ackley functions and their gradients.
- Implement Gradient Descent, SGD with Momentum, RMSprop, Adagrad and Adam from scratch.
- Compare the optimizers at learning rates 0.01, 0.05 and 0.1.
- Study convergence behaviour and record the final objective value, iterations and time.
- Use the results to identify which methods performed best for the two functions.

2. Functions Used

2.1 Rosenbrock Function

The function used was:

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$

Its global minimum is at (x, y) = (1, 1), where f(x, y) = 0. The starting point used for the main experiment was (−1.5, 2.0).

2.2 Simplified 1D Ackley Function

$$f(x) = -20\exp(-0.2\sqrt{(x^2)}) - \exp(\cos(2\pi x)) + 20 + e$$

2.3 Optimization Algorithms

Optimizer	Working principle
Gradient Descent	Updates parameters directly in the opposite direction of the current gradient.
Momentum	Maintains a velocity term that accumulates gradients and can accelerate movement along consistent directions.
Adam	Uses first- and second-moment estimates with bias correction, giving parameter-wise adaptive step sizes.
RMSprop	Scales the gradient using a moving average of squared gradients.
Adagrad	Accumulates squared gradients to adapt the effective learning rate over time.

3. Experimental Methodology

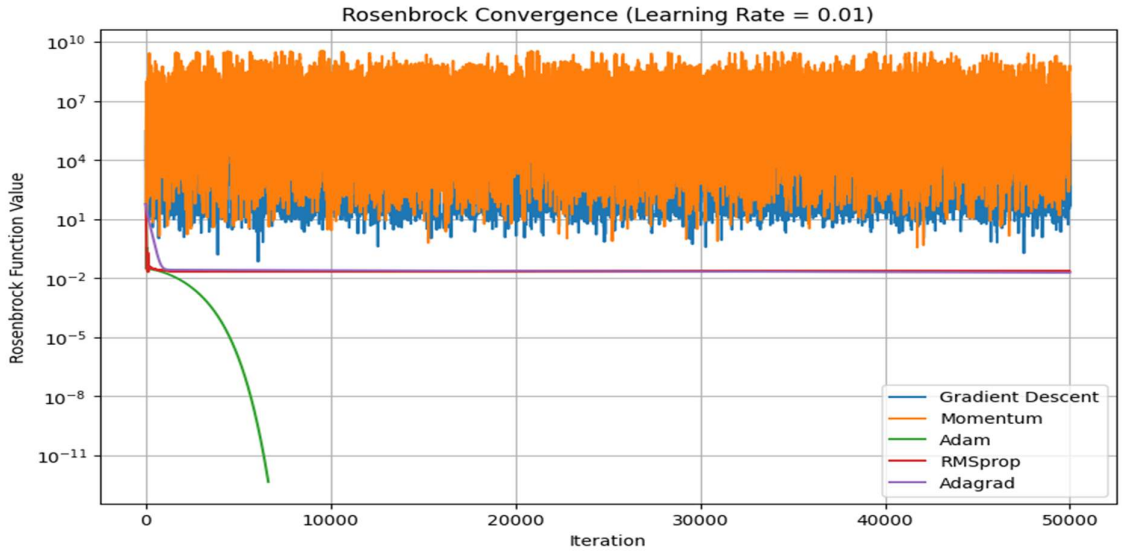
Setting	Value
Learning rates	0.01, 0.05, 0.10
Maximum iterations	50,000
Stopping tolerance	1e-8
Gradient clipping threshold	1000
Initial Rosenbrock point	[0.5, 1]
Initial Ackley point	[4]

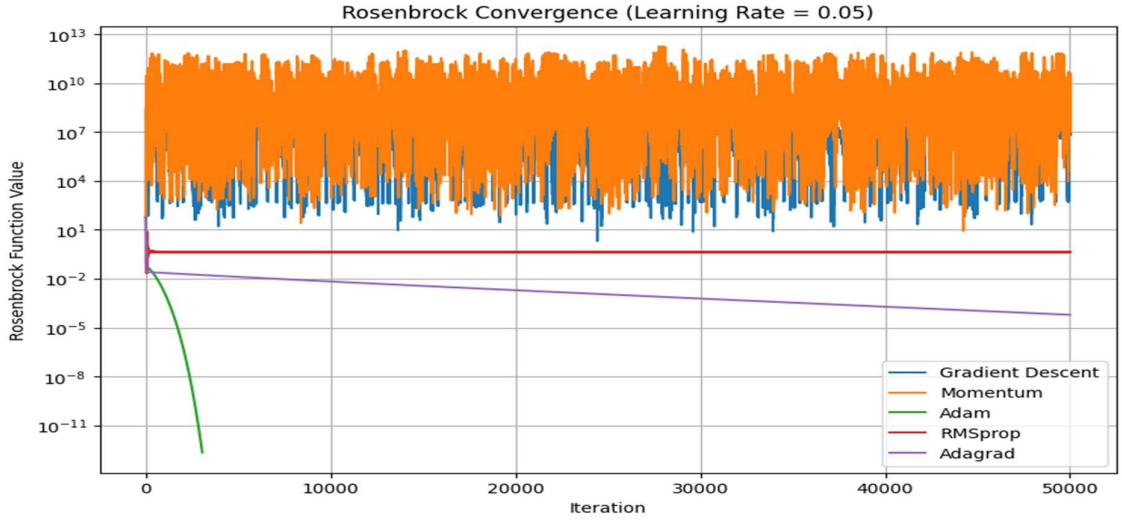
4. Experimental Results

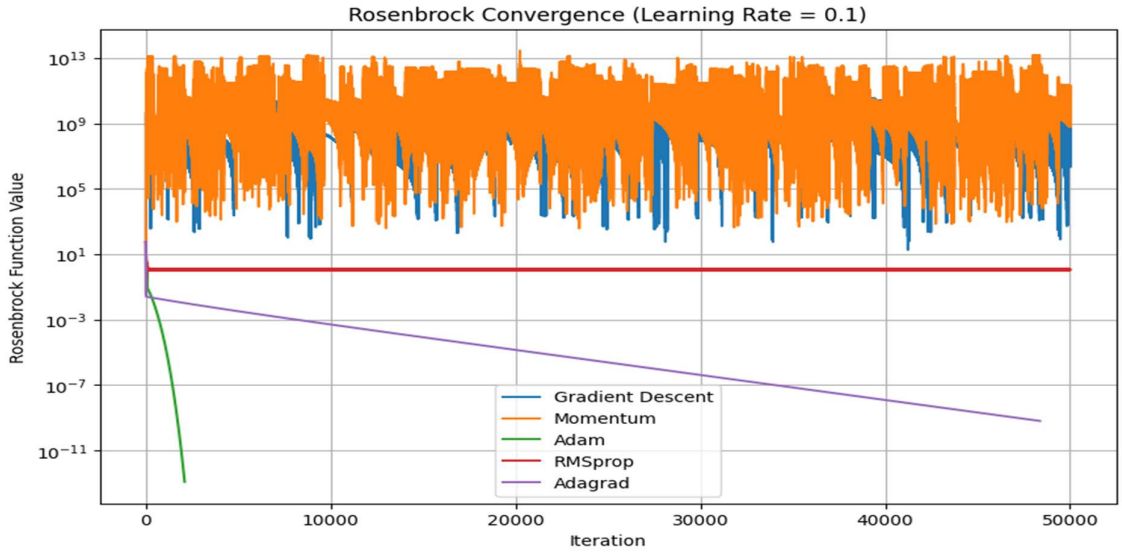
4.1 Rosenbrock Function experiment outcomes on different learning rates

	Learning Rate	Optimizer	Final x	Final y	Final f(x,y)	Iterations	Time (seconds)	Status
0	0.01	Gradient Descent	-6.387857	10.497436	9.190767e+04	50000	2.671795	Maximum iterations
1	0.01	Momentum	28.103848	14.843196	6.006061e+07	50000	2.486498	Maximum iterations
2	0.01	Adam	0.999999	0.999999	4.407535e-13	6625	0.376658	Converged
3	0.01	RMSprop	0.980149	0.975543	2.245074e-02	50000	2.111165	Maximum iterations
4	0.01	Adagrad	0.863607	0.745268	1.863315e-02	50000	1.894554	Maximum iterations
5	0.05	Gradient Descent	-66.676901	528.506268	1.534531e+09	50000	1.824791	Maximum iterations
6	0.05	Momentum	-59.228748	489.803610	9.109815e+08	50000	2.034977	Maximum iterations
7	0.05	Adam	1.000000	0.999999	2.165522e-13	3050	0.148078	Converged
8	0.05	RMSprop	0.675000	0.515625	4.656250e-01	50000	3.386350	Maximum iterations
9	0.05	Adagrad	0.992162	0.984356	6.151895e-05	50000	2.174506	Maximum iterations
10	0.10	Gradient Descent	-30.382577	779.949548	2.050218e+06	50000	1.817085	Maximum iterations
11	0.10	Momentum	221.717617	2871.313438	2.142523e+11	50000	2.080752	Maximum iterations
12	0.10	Adam	1.000000	0.999999	1.181594e-13	2095	0.127680	Converged
13	0.10	RMSprop	0.300000	0.015000	1.052500e+00	50000	2.095906	Maximum iterations
14	0.10	Adagrad	0.999975	0.999949	6.442609e-10	48374	1.826817	Converged

4.2 Rosenbrock Convergence Graphs on Different Learning Rates



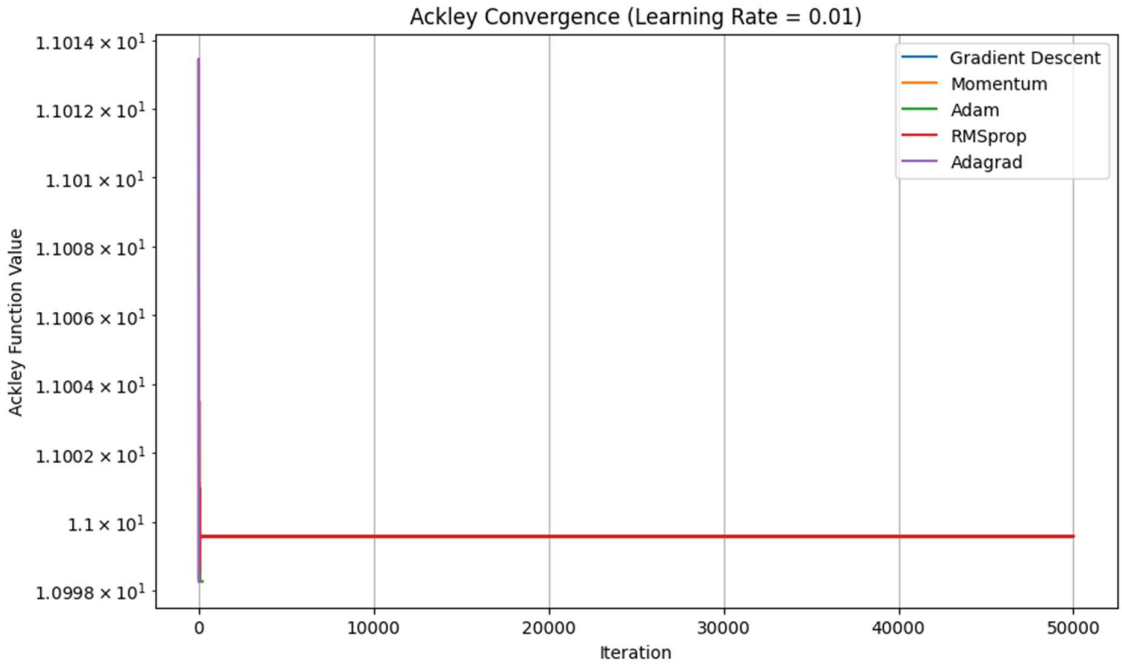


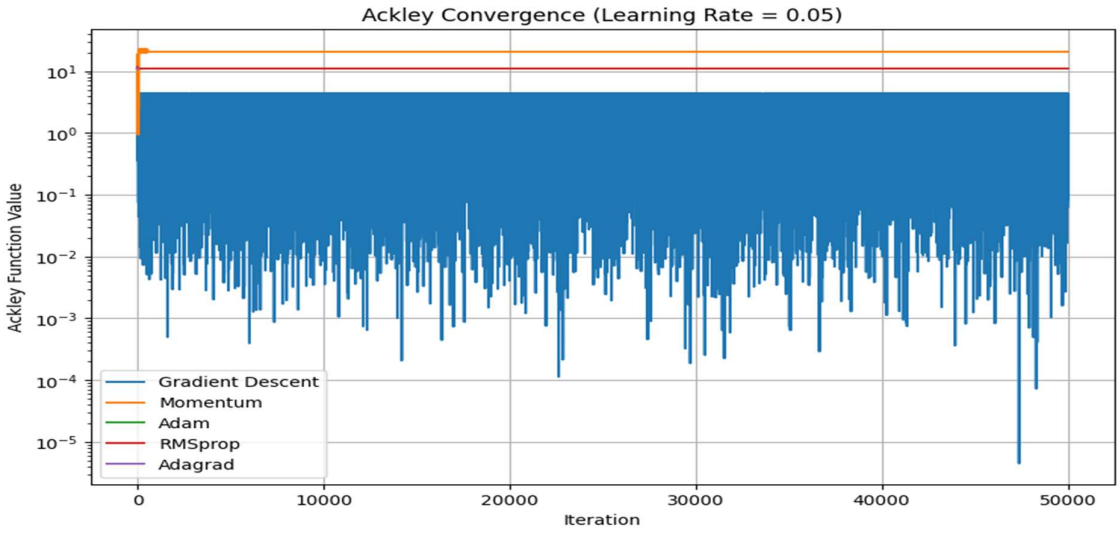


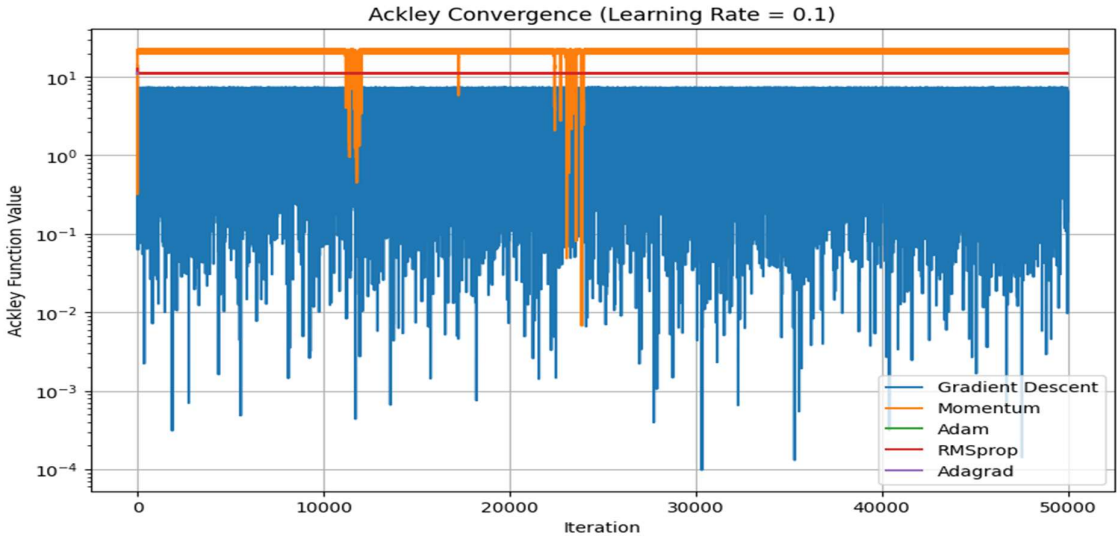
4.3 Ackley Function experiment outcomes on different learning rates

	Learning Rate	Optimizer	Final x	Final f(x)	Iterations	Time (seconds)	Status
0	0.01	Gradient Descent	3.983068	10.998262	6	0.000646	Converged
1	0.01	Momentum	3.983065	10.998262	168	0.015630	Converged
2	0.01	Adam	3.983067	10.998262	209	0.023213	Converged
3	0.01	RMSprop	3.988034	10.999557	50000	3.064596	Maximum iterations
4	0.01	Adagrad	3.983068	10.998262	19	0.001915	Converged
5	0.05	Gradient Descent	0.236179	2.550432	50000	2.171394	Maximum iterations
6	0.05	Momentum	73.883861	20.611076	50000	2.353683	Maximum iterations
7	0.05	Adam	3.983068	10.998262	240	0.026523	Converged
8	0.05	RMSprop	4.007193	11.029113	50000	3.910822	Maximum iterations
9	0.05	Adagrad	3.983068	10.998262	17	0.001979	Converged
10	0.10	Gradient Descent	0.991967	3.602515	50000	2.347533	Maximum iterations
11	0.10	Momentum	555.190338	21.276111	50000	2.560812	Maximum iterations
12	0.10	Adam	3.983067	10.998262	222	0.019531	Converged
13	0.10	RMSprop	3.929165	11.137523	50000	2.679675	Maximum iterations
14	0.10	Adagrad	3.983068	10.998262	25	0.002460	Converged

4.4 Ackley Convergence Graphs on different learning rates







5. Conclusion

- **Rosenbrock:** Adam with **learning rate 0.1** was the best-performing configuration.
- **Ackley:** Gradient Descent with **learning rate 0.05** reached the lowest function value, but did not converge within the iteration limit.
- For **stable convergence on Ackley**, **learning rate 0.01** performed better.
- Higher learning rates caused instability for standard Gradient Descent and Momentum, while Adam maintained stable convergence.

Task 2 : Implementing Linear Regression Using a Multi-Layer Neural Network from Scratch

1. Objectives

- Build a neural network regression model from scratch using NumPy.
- Use ReLU activation in hidden layers and a linear output layer for regression.
- Implement forward propagation and backpropagation manually.
- Compare Gradient Descent, Momentum, and Adam optimizers.
- Study the effect of learning rates 0.01 and 0.001.
- Evaluate a deeper neural-network architecture as an additional experiment.
- Study the effect of L2 regularization with $\lambda = 0.001$.
- Compare models using test MSE and RMSE.

2. Dataset

The California Housing dataset was loaded using sklearn.datasets.fetch_california_housing. The complete dataset contains 20,640 observations and 9 columns. Eight columns describe housing and geographical characteristics, while MedHouseVal is the target variable.

Feature	Role	Description
MedInc	Input	Median income
HouseAge	Input	Median house age
MedHouseVal	Target	Median house value

For the neural-network experiment, only two input features—MedInc and HouseAge—were selected. The target was MedHouseVal. This gives X with shape (20640, 2) and y with shape (20640, 1). The data were split into 80% training (16,512 samples) and 20% testing (4,128 samples).

3. Data Preprocessing

The training data were standardized using the training-set mean and standard deviation. The same training statistics were then applied to the test data. This prevents information from the test set from influencing the normalization process and places the input features on comparable scales.

4. Neural Network Architecture

- The required network contains two input features, two hidden layers, and one regression output:

$$2 \rightarrow 5 \rightarrow 3 \rightarrow 1$$

- ReLU is used in the hidden layers. The final layer is linear because the task is regression.
- The implementation stores weights and biases for every layer and performs matrix-based forward propagation. The loss is Mean Squared Error (MSE).

5. Backpropagation

Backpropagation computes gradients of the MSE loss with respect to every weight and bias. The ReLU derivative is applied to hidden-layer pre-activations. Optional L2 regularization adds a penalty proportional to the squared weights.

Three optimizers were implemented:

- Gradient Descent – updates parameters directly using the negative gradient.
- Momentum – maintains velocity accelerate movement in consistent gradient directions.
- Adam – maintains first and second moment estimates of gradients for adaptive updates.

Each configuration was trained for 5,000 epochs. The experiments used learning rates of 0.01 and 0.001 with a fixed random seed of 10 for reproducibility.

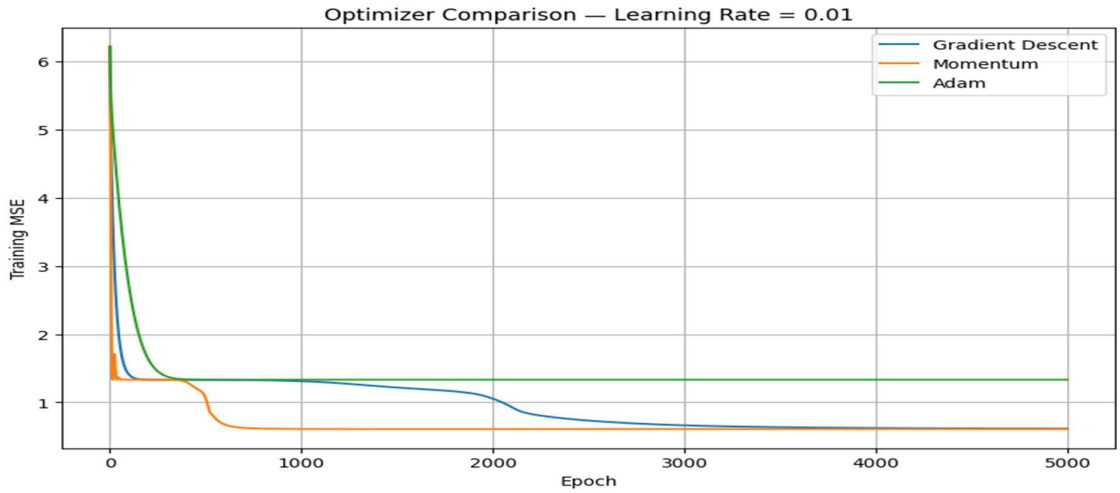
6. Experimental Results

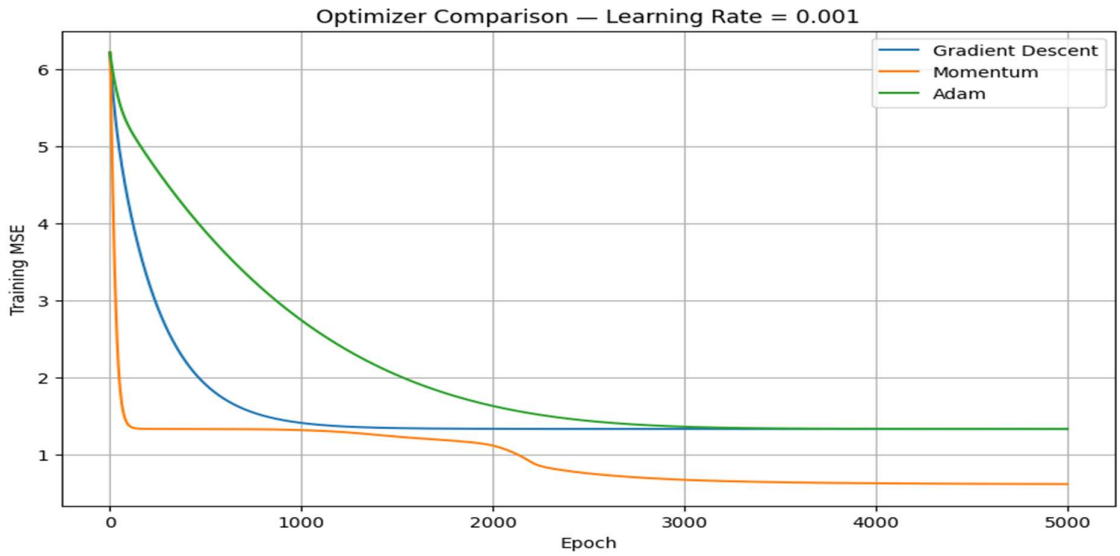
The required 2→5→3→1 architecture was trained with all optimizer and learning-rate combinations.

	Optimizer	Learning Rate	Final Training Loss	Test MSE	Test RMSE	Training Time (seconds)	Epochs
0	Momentum	0.010	0.608810	0.608873	0.780303	15.076244	5000
1	Gradient Descent	0.010	0.614233	0.611663	0.782089	16.402199	5000
2	Momentum	0.001	0.614639	0.611906	0.782244	16.485480	5000
3	Gradient Descent	0.001	1.329611	1.323679	1.150512	14.921326	5000
4	Adam	0.001	1.332634	1.327218	1.152050	16.991086	5000
5	Adam	0.010	1.332633	1.327220	1.152050	15.937334	5000

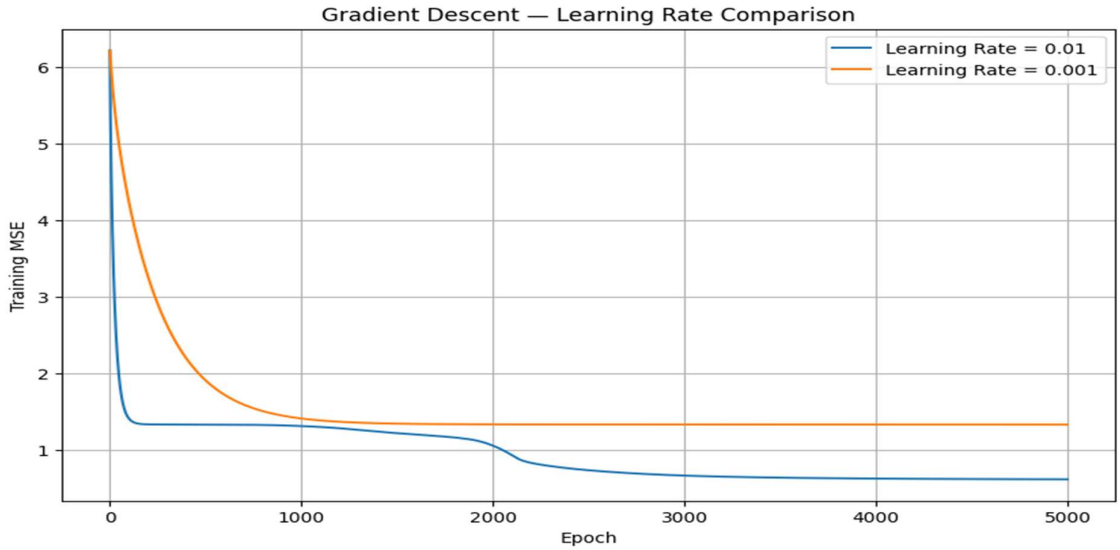
Best required-architecture configuration: Momentum with learning rate 0.01. It achieved the lowest test MSE of 0.608873 and RMSE of 0.780303 among the required experiments. The results also show that learning rate 0.01 was more effective than 0.001 for Gradient Descent and Adam in this setup.

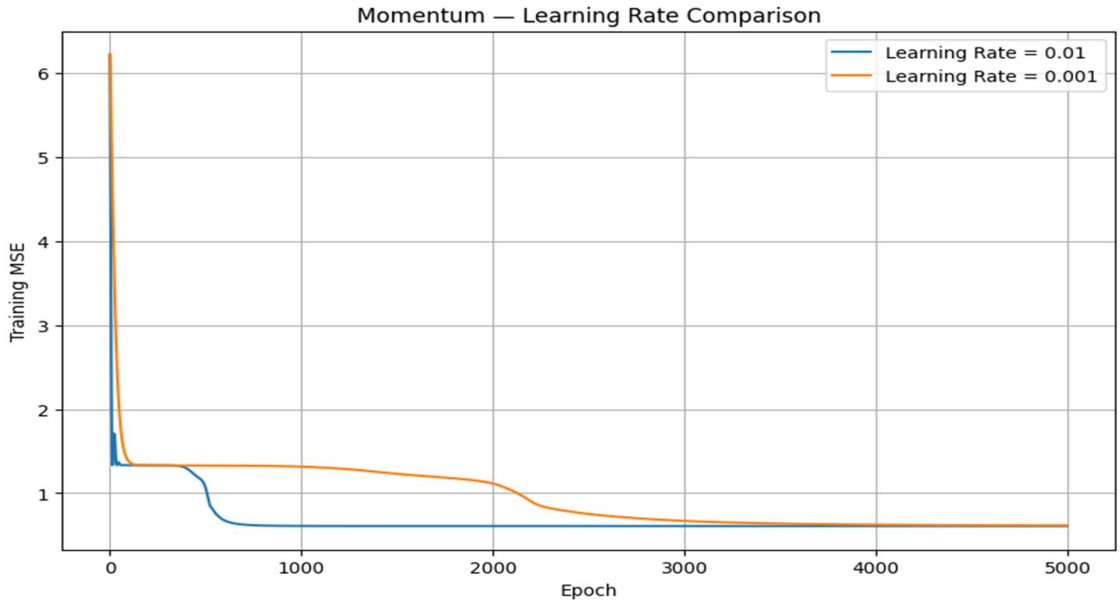
6.1 Mean Squared Error vs Epochs of Optimizers on different learning rates

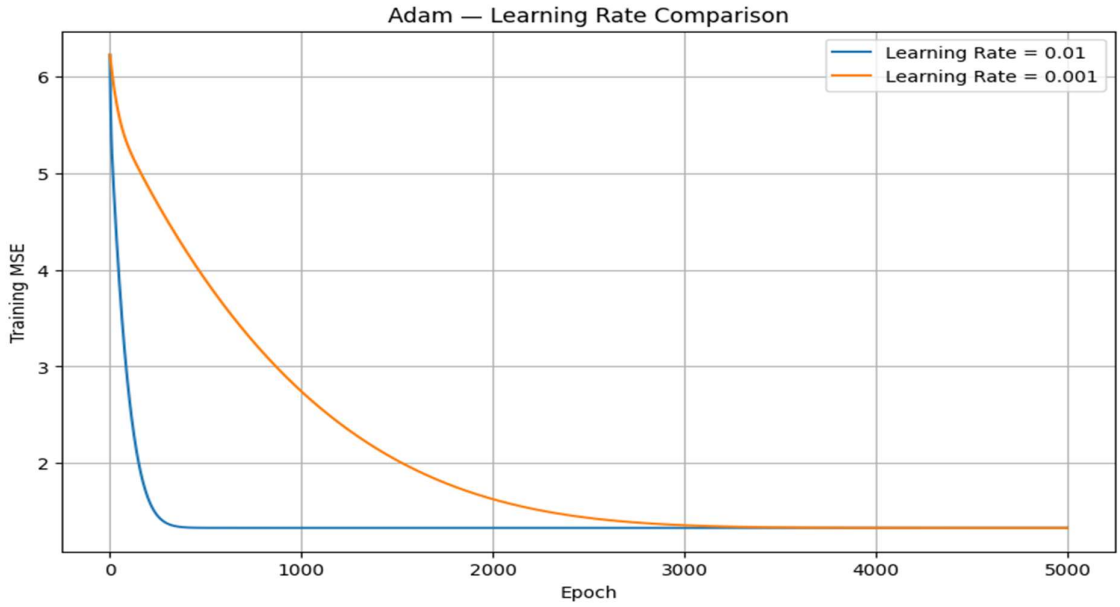




6.2 Optimizers comparison on different learning rates

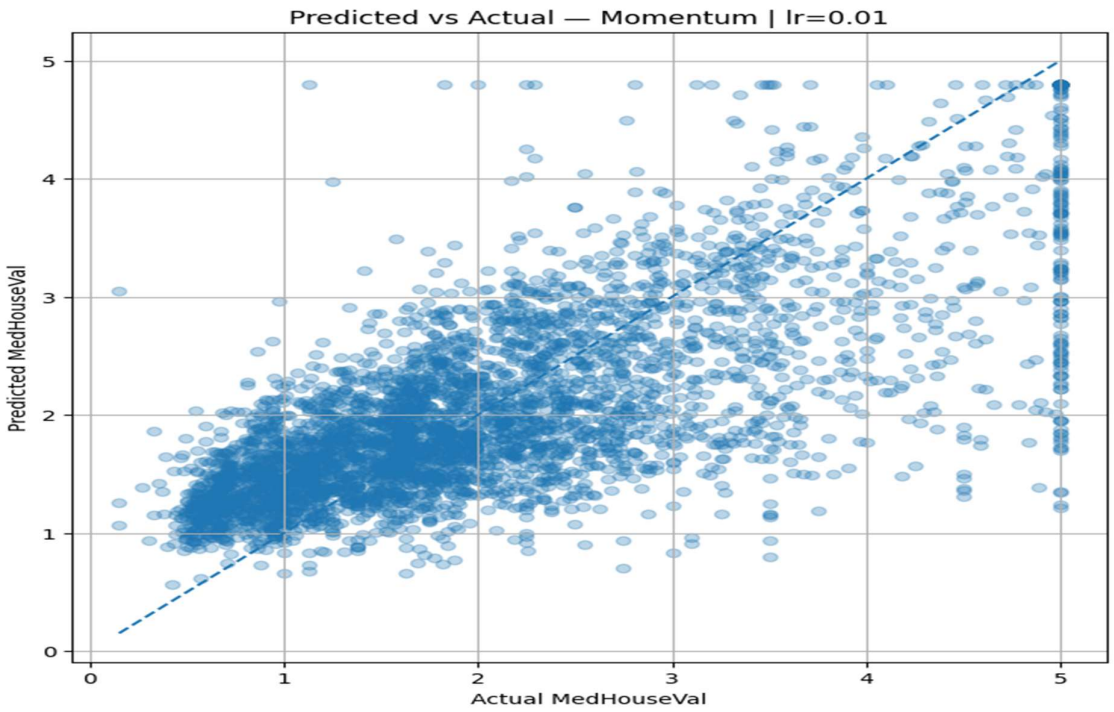






- Gradient Descent: Changing the learning rate from 0.001 to 0.01 reduced test MSE from 1.323679 to 0.611663.
- Momentum: The same change reduced test MSE from 0.611906 to 0.608873.
- Adam: It did not converge to as compare to others, its test MSE remained around 1.327.

Therefore, for the required architecture, the best learning rate is 0.01, with Momentum giving the strongest overall result.



7. Bonus Experiment 1 – Additional Hidden Layer

The required 2 → 5 → 3 → 2 → 1 architecture was trained with all optimizer and learning-rate combinations.

	Optimizer	Learning Rate	Final Training Loss	Test MSE	Test RMSE	Training Time (seconds)	Epochs
0	Momentum	0.010	0.604960	0.604961	0.777793	20.985322	5000
1	Adam	0.001	0.606597	0.607419	0.779371	21.894361	5000
2	Gradient Descent	0.010	0.621940	0.621257	0.788198	21.277971	5000
3	Gradient Descent	0.001	1.311595	1.301765	1.140949	19.940159	5000
4	Momentum	0.001	1.332633	1.327222	1.152051	20.079673	5000
5	Adam	0.010	1.332632	1.327231	1.152055	21.912400	5000

- The best configuration for both architectures was Momentum with learning rate 0.01.

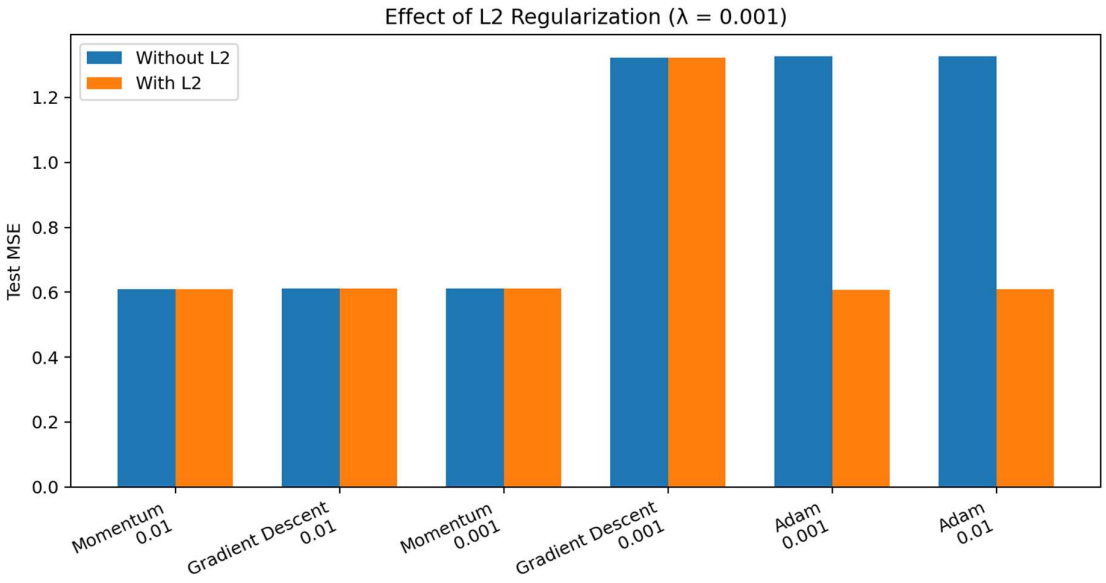
	Architecture	Best Optimizer	Learning Rate	Test MSE	Test RMSE
0	2 → 5 → 3 → 1	Momentum	0.01	0.608873	0.780303
1	2 → 5 → 3 → 2 → 1	Momentum	0.01	0.604961	0.777793

- The deeper model improved test MSE from 0.608873 to 0.604961 and RMSE from 0.780303 to 0.777793. Thus, the additional hidden layer provided a small but measurable improvement.

8. Bonus Experiment 2 – L2 Regularization

L2 regularization was introduced with $\lambda = 0.001$.

	Configuration	MSE Without L2	MSE With L2	RMSE Without L2	RMSE With L2
0	Momentum | lr=0.01	0.608873	0.609130	0.780303	0.780468
1	Gradient Descent | lr=0.01	0.611663	0.611823	0.782089	0.782191
2	Momentum | lr=0.001	0.611906	0.612113	0.782244	0.782376
3	Gradient Descent | lr=0.001	1.323679	1.323741	1.150512	1.150539
4	Adam | lr=0.001	1.327218	0.607028	1.152050	0.779120
5	Adam | lr=0.01	1.327220	0.609180	1.152050	0.780500



- Adam at learning rate 0.001 achieved the lowest test MSE among the regularized runs. This is a notable improvement over Adam without regularization, whose test MSE was approximately 1.327.
- However, for the overall best model, the deeper Momentum configuration without L2 remained slightly better at MSE 0.604961.